

AI Is Not Enough: A Hybrid Technical Approach to AI Adoption in UI Linting With Heuristics

Yuwen Lu*
University of Notre Dame
Notre Dame, IN, USA
ylu23@nd.edu

Jamie Blass
Google
Asheville, NC, USA
jamieblass@google.com

Tiffany Kneareem
Google
Cambridge, MA, USA
tkneareem@google.com

Clara Kliman-Silver
Google
San Francisco, CA, USA
klimansilver@google.com

Shona Dutta
Google
San Francisco, CA, USA
shonad@google.com

Frank Bentley
Google
San Francisco, CA, USA
fbentley@google.com

ABSTRACT

Design systems have become an industry standard for creating consistent, usable, and effective digital interfaces. However, detecting and correcting violations of design system guidelines, known as UI linting, is a major challenge. Manual UI linting is time-consuming and tedious, making it a prime candidate for automation. This paper presents a case study of adopting AI for UI linting. Through collaborative prototyping with UX designers, we analyzed the limitations of existing AI models and identified designers' core needs and priorities in UI linting. With such knowledge, we designed a hybrid technical pipeline that combines the deterministic nature of heuristics with the flexibility of large language models. Our case study demonstrates that AI alone is not sufficient for practical adoption and highlights the importance of a deep understanding of AI capabilities and user-centered design approaches.

CCS CONCEPTS

• Human-centered computing → Systems and tools for interaction design.

KEYWORDS

user interface (UI) design, UI linting, design systems, artificial intelligence, large language models

ACM Reference Format:

Yuwen Lu, Tiffany Kneareem, Shona Dutta, Jamie Blass, Clara Kliman-Silver, and Frank Bentley. 2024. AI Is Not Enough: A Hybrid Technical Approach to AI Adoption in UI Linting With Heuristics. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3613905.3637135>

*Work done as a student researcher at Google.

1 INTRODUCTION

The user experience (UX) industry has seen significant growth in the past years. Technology designers and developers have become increasingly focused on UX, ensuring products are user-friendly and usable. Over the past decade, design systems have become the standard way that companies manage design and ensure standardization across diverse apps created by different teams [8]. Design systems are created to manage visual and interaction design with a systematic, or system-level, approach. They usually contain a set of user interface (UI) components and can offer guidance for layout, visual aesthetics, and user task flows. They may also provide advice for product content, branding, and tone [30]. Adopting design systems helps teams to easily maintain the cohesiveness and consistency of digital products' visual appearances and user experiences at a large scale. While design systems such as Google Material Design¹ have gained large-scale success in the UX industry, they are still an under-explored topic in academia [8, 10, 26, 39]. In this work, we focus on **adherence** to the guidelines in design systems and specifically investigate the **detection of guideline violations**.

As teams adopt design systems to create digital products, a main challenge is to understand how well apps are adhering to the design system guidelines [21, 41]. The centralized nature of design systems can cause maintenance overhead and violations in adoption [10]. Specifically, violations of design system guidelines and component specifications can break the consistency of user experiences, defeating the purpose of design systems [39]. In practice, such violations come from product teams' unique situations and needs: edge cases where the desirable component is not available, component customization for specific purposes, or simply negligence of design systems. Manually linting design system violations take up a great amount of designers' time and effort.

UI linting, i.e. checking UI against design guidelines, has been proposed as a solution to identify guideline violations. It has also been referred to as "design violation check" or "design smell detection" in previous literature [21, 39]. UI linting is adjacent to software testing [4, 29, 35], but has a different focus on interaction and visual design pattern violations rather than implementation errors. Design systems typically include guidelines of different levels, from low-level specifications of component attribute values (e.g. typography size and component sizing) to high-level guidelines such

¹<https://material.io>

as UX writing best practices and component selection in different scenarios [39].

Recent research on automatic UI linting has demonstrated its technical feasibility through two main approaches: *heuristic-based* and *AI-enabled*. In *heuristic-based* linting, researchers handcraft deterministic validation rules based on design guidelines, and check UI component attributes against these rules [39]. However, crafting validation rules for large design systems can take significant effort and are hard to scale up. In addition, heuristics are more suitable for specific attribute linting (e.g. checking the size, padding, and color of UI components) than guidelines that require more contextual information (e.g. when displaying both destinations and actions, bottom app bars are more preferred than bottom navigation bars)². In comparison, *AI-enabled* linting techniques use mostly deep learning (DL) models that are trained on large UI datasets [41]. Compared to deterministic heuristics, AI models can be more flexible and comprehensive regarding the types of guidelines, but often suffer from being less deterministic and interpretable. Given the amount of data required to train DL models, such approaches are feasible only for larger design teams with many design instances to work with.

In this work, we explore the real-world adoption of AI models in UI linting inside a large technology company. Working closely with UX designers, we identified the challenges and opportunities to integrate AI support in UI linting. We also summarized designers' three unique needs in UI linting: tight alignment with design system guidelines, high precision in detailed evaluation, and clear explanation of violations for smooth handoff. Through this case study, we share how we started by assuming AI models as our ideal solution, then gradually understood its constraints in practice, and landed on a more nuanced, hybrid solution involving both heuristics and AI. We present our design of a *hybrid* technical pipeline for UI linting, to utilize both the specificity of heuristics and the flexibility of AI. We explain our design rationale in detail, providing a carefully designed, AI-enabled technical pipeline that bridges the latest capabilities of large language models (LLMs) with designers' practical needs.

2 DESIGN EXPLORATIONS AND CONSIDERATIONS

We initiated a multi-day, in-person workshop with designers and UX researchers to understand current linting practices and needs, aiming to gather user requirements for a linter that checks for design system guideline violations. The results of this workshop were a list of critical requirements for the linter from designers' perspectives:

- (1) The linter should include the *screenshots* and *explanations* for detected design system guidelines violations;
- (2) The linter's successful completion rate is at least 90%. A successful completion is defined as the tool outputting an analysis that does not require additional human/manual intervention to complete the analysis. In other words, the linter's 'confidence rate' is scored as 95%+ on at least 90% of the screenshots analyzed;

- (3) The linter should calculate and publish a 'confidence' level to signify the accuracy of results;
- (4) The linter should be able to aggregate analysis from multiple apps and give insights into common patterns of proper use and violations.

Other requirements include that the linter has to work across multiple screen sizes and should be used by designers with little engineering experience. Generally, not only does the linter need to be able to correctly identify UI components, it also needs to do so with a high level of accuracy to ensure that the linter's comparisons against Material guidelines can be trusted.

With a basic understanding of designers' needs for automation support in UI linting, we collaboratively explored a few potential technical solutions for the linter through follow-up interviews with UX designers. For each potential solution, we mocked up and presented a quick prototype to elicit designers' feedback. The goal of these mock-ups and prototypes is two-fold: first, to test out the technical feasibility of these solutions and find out their alignment and limitations regarding designers' needs; second, to probe designers with potential usage scenarios to further understand their priorities and pain points. Below, we describe these potential solutions, pinpoint their respective advantages and restrictions based on designer feedback, and report how well each one can meet the needs of designers.

2.1 Potential Technical Solutions

Firstly, we explored potential technical solutions by considering the latest advancements in the fields of UI linting [39, 41], UI understanding with AI models [6, 7, 17, 31, 32, 36, 37, 40], and large language/vision-language models [11, 12, 15, 24, 34]. We also referred to the latest guidelines and specifications in Material Design³ because it is our target design system. This orientation from the literature gave us a working understanding of various approaches that we could try.

2.1.1 Design guideline understanding with LLMs. Large language models (LLMs) have significantly improved model performance in many text-based understanding and generation tasks [5]. Specifically, past research has demonstrated LLMs' general ability in UI understanding through few-shot learning with text-only view hierarchy UI representations [31].

Given that most of design system information is encapsulated in text, we investigated the potential of design guideline understanding with LLMs (our final pipeline design that utilizes LLMs can be found in Fig. 2). Information about specific design systems does not constitute a large portion of most LLMs' training data, so including information regarding design guidelines in prompts can support more effective interactions with LLMs and reduce hallucination [31]. However, large design systems can contain numerous guidelines, which are hard to fit into one single context window (usually from a few thousand tokens up to 100K).

The main limitation of using LLMs is the constraint of only using text-based UI representations (i.e. implementation code or view hierarchies). No information can be included in the analysis with LLMs, which limits the effectiveness and accuracy of linting

²<https://m3.material.io/components/bottom-app-bar/guidelines>

³<https://material.io>

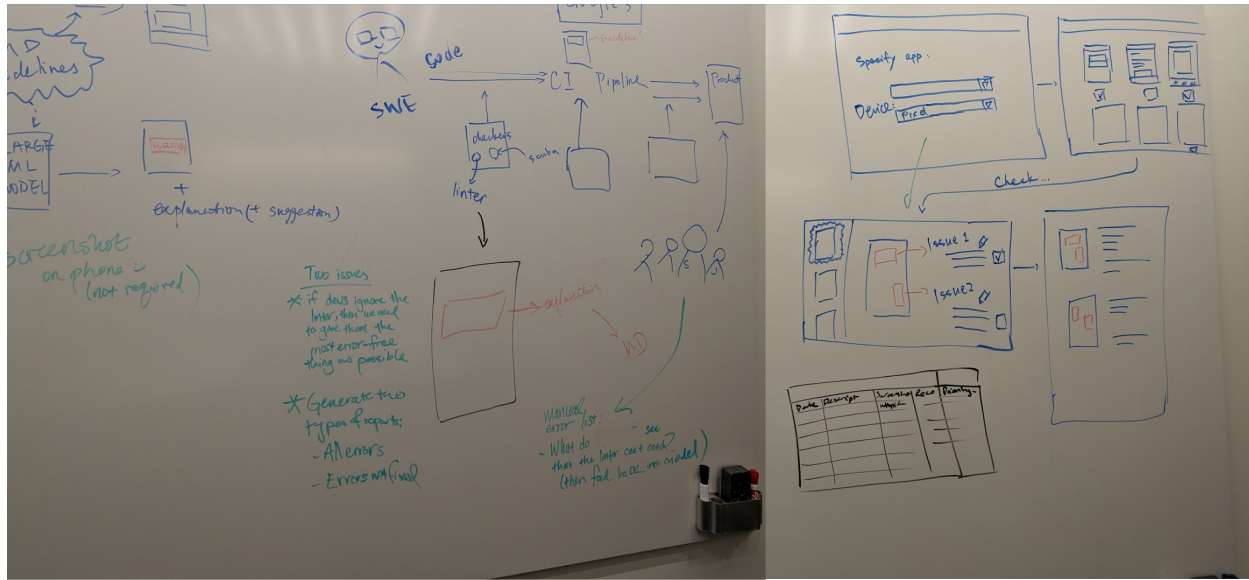


Figure 1: Examples of linter interface mock-ups and usage scenario sketches we produced during our workshop.

results. Thus, we also considered using vision-language models for our task.

2.1.2 Vision-based linting with large vision-language models. Recent advancements in large vision-language models have demonstrated great promise in visual understanding with AI models [38]. Given the multi-modal nature of UI data [17, 31, 32] (i.e. UIs can be represented as screenshot images and/or implementation code), fine-tuning a pre-trained large vision-language model with specific UI guideline violation datasets is an obvious candidate for UI linting.

However, a few constraints emerged after trying to align this approach with UX designers' practical working processes and desires:

Lack of high precision in violation detection. UI linting in practice requires high precision regarding component attributes, such as position and sizing, where large models can often make mistakes [13, 18, 33]. A violation by a few pixels in spacing or alignment can affect the information density on screens, consequently harming components' usability [1].

Potential misalignment with guidelines through hallucination. UI linting requires *strong alignment* with design guidelines. The potential of hallucination in large vision-language models introduces a level of uncertainty for designers to work with AI in UI linting [13, 18, 33]. Identifying and correcting mistakes by AI, where the identified violation is inconsistent with design guidelines, is likely more laborious than the pure manual process [14].

Scarcity of design violation datasets. A more practical constraint, for both working with language and vision-language models, is the scarcity of design violation datasets. Collecting UI datasets itself already requires much effort [9]; having a decent-sized labeled

dataset on design violations requires the labelers to have design expertise and a good familiarity with design guidelines. While self-supervised learning [2, 17, 41] and synthetic data [3, 27, 28, 41] showed potential, they are likely not adequate for real-world production needs.

2.1.3 Algorithmic, pixel-level comparison. We used pixel-level matching algorithms to conduct pairwise comparisons between a component under test and the standard component from guidelines. Specifically, we segmented UI components from the screenshots and used the SIFT [20] and ORB [22] image matching algorithms to determine the difference between the two components. The underlying rationale was to automatically identify UI components that are significantly different from design guidelines, so that designers can target these elements for linting. Such an approach can be easily scalable for large design systems. Compared to large vision-language models, this approach can scrutinize at a more granular level and meet the level of precision designers desire in UI linting.

We prototyped the solution in Google Colab⁴ and presented image matching algorithm results to designers for feedback. Our results listed the component under test and the corresponding component from Material Design side-by-side, together with the algorithm prediction result of whether the two components are similar enough (True/False), indicating guideline compliance.

Designers' biggest critique of this approach is that it does not explain why or how a design violates a guideline. Identifying the specifics of guideline violation, including pinpointing relevant guidelines, measuring the specific differences, and suggesting correction options, takes up the majority of designers' manual efforts.

⁴<https://colab.google>

Thus, simply labeling potential violation areas does not address designers' most urgent needs and provides only limited help.

2.2 Summary of Designer Needs (DNs)

Through prototyping and gathering feedback with our technical solutions above, we probed designers and revealed their main goals and needs in UI linting support. We summarize them into the 3 main themes below.

2.2.1 DN1: Evidence-based detection that aligns with design guidelines. Adhering to design guidelines is as essential for UI linting as it is, if not more so, for UI design. Explaining how a violation deviates from design guidelines is at the core of linting practices. If AI predictions of such explanations contain conflicting information against design guidelines, it is hard to identify and repair, causing great frustration in linting. If the cost of such repairment exceeds the perceived benefits, designers will likely return to the more familiar manual linting process [14]. Thus, ensuring the linting results correspond to design system guidelines is of high priority for our linter design.

2.2.2 DN2: High precision for pixel-level scrutiny. Many design guideline violations happen at a granular level regarding aspects including element sizing, positioning, and coloring. Sometimes, they can be hard to identify without extensive training and experience in design. Designers emphasized that detecting violations of design guidelines requires a high level of precision. Otherwise, detected violations or improvement suggestions that contain inaccurate sizing values or color tokens can create friction and confusion. This significantly hurts any UI linters' practical utility.

Based on designer feedback, interestingly, predicting areas of interest for design violations is often not helpful enough. Experienced designers can locate these issues fairly quickly, thus the automation of this process would provide only limited assistance. In contrast, measuring the exact violation specifics (e.g. color and pixel values) takes up most of the manual, tedious linting efforts for designers. Designers reported that reducing such manual measuring efforts and providing evidence of violations (e.g. specific values of component attributes) is a top priority for linter support.

It is worth noticing that this design need applies more towards low-level design attribute specifications than higher-level, general guidelines. These are also notably more challenging to execute than high-level guideline checks, as the meticulous measurement and examination of detailed attributes is a significantly laborious process.

2.2.3 DN3: Clear explanation of violation for smooth handoff. Connected to DN2, the final goal of UI linting is to provide feedback to product teams and guide them to align their design with guidelines. This handoff process calls for clear communication and precise descriptions of violations. Otherwise, confusion and miscommunication might take place, which can further hurt collaborations with other teams and reduce design system teams' credibility. Ideally, AI solutions can automate the whole process and produce well-documented violation descriptions and improvement suggestions. However, most existing deep-learning-based UI violation detection models only predict UI consistency without explanations [25, 41], limiting their utility in real-world scenarios.

3 OUR SOLUTION: A HYBRID TECHNICAL PIPELINE

Our explorations and past research have demonstrated the respective pros and cons of heuristic-based and AI-based linting approaches. On one hand, heuristic-based approaches can support granular scrutiny, but lack flexibility and scalability for large design systems. On the other hand, existing AI models are flexible and can be easily scaled up, but can be inaccurate and unable to provide detailed violation explanations. Merging these two approaches would offset each other's limitations, harnessing the precise analysis of heuristic-based linting with the scalability and flexibility of AI-driven systems.

Based on the designer needs we identified in the previous section, we designed an attribute-based, hybrid⁵ technical pipeline that both utilizes the accuracy of heuristics and the flexibility and scalability of AI models. Specifically, we used heuristics to extract the relevant attributes from target UI screens' view hierarchies, combined them with the retrieved relevant guidelines from the design system documentation, and leveraged LLMs as reasoning engines to evaluate the design compliance of the target UI.

The designed pipeline contains three main modules: (a) heuristics-based extraction of UI component attributes, (b) relevant guideline retrieval from design system documentation, and (c) linting evaluation with LLMs.

The main purpose of module (a) is to support the high precision for pixel-level scrutiny needed in UI linting (DN2). While vision-based AI models can perceive general contents and styles of a screen [17, 32], they cannot measure detailed pixel values of UI components, especially when image transformations are performed and precise pixel information is lost. By extracting the relevant attributes for different component types using pre-defined heuristics, we expose components' granular and detailed information to LLMs for evaluation. This goes a level deeper than image-based AI evaluations. The heuristics for attribute extraction depend on linting priorities (e.g. common violations that significantly impact user experiences) as well as the design system specifications. For example, for bottom navigation bars in Material Design, the measurement specifications contain specific values for container width, height, shape, icon shape, etc. We extract these attributes from the view hierarchies for evaluation.

In module (b), we extract the relevant guidelines from target design systems for evaluation. This follows the paradigm of retrieval-based generation (RAG) [16], common for knowledge-intensive tasks using pre-trained LLMs. LLMs likely did not obtain accurate knowledge of specific design systems through pre-training. By using RAG, we supplement design system information in the prompts. This helps to align evaluation results with specific design systems and provides flexibility for all design system teams. Specifically, RAG can be achieved by storing all design system documentation in a vector database and using embedding-based retrieval techniques. Existing tools like LangChain⁶ have incorporated support for such operations.

⁵We use the term "hybrid" as a reference to the combined usage of the *heuristics-based* and *AI-enabled* techniques in our pipeline.

⁶<https://www.langchain.com/>

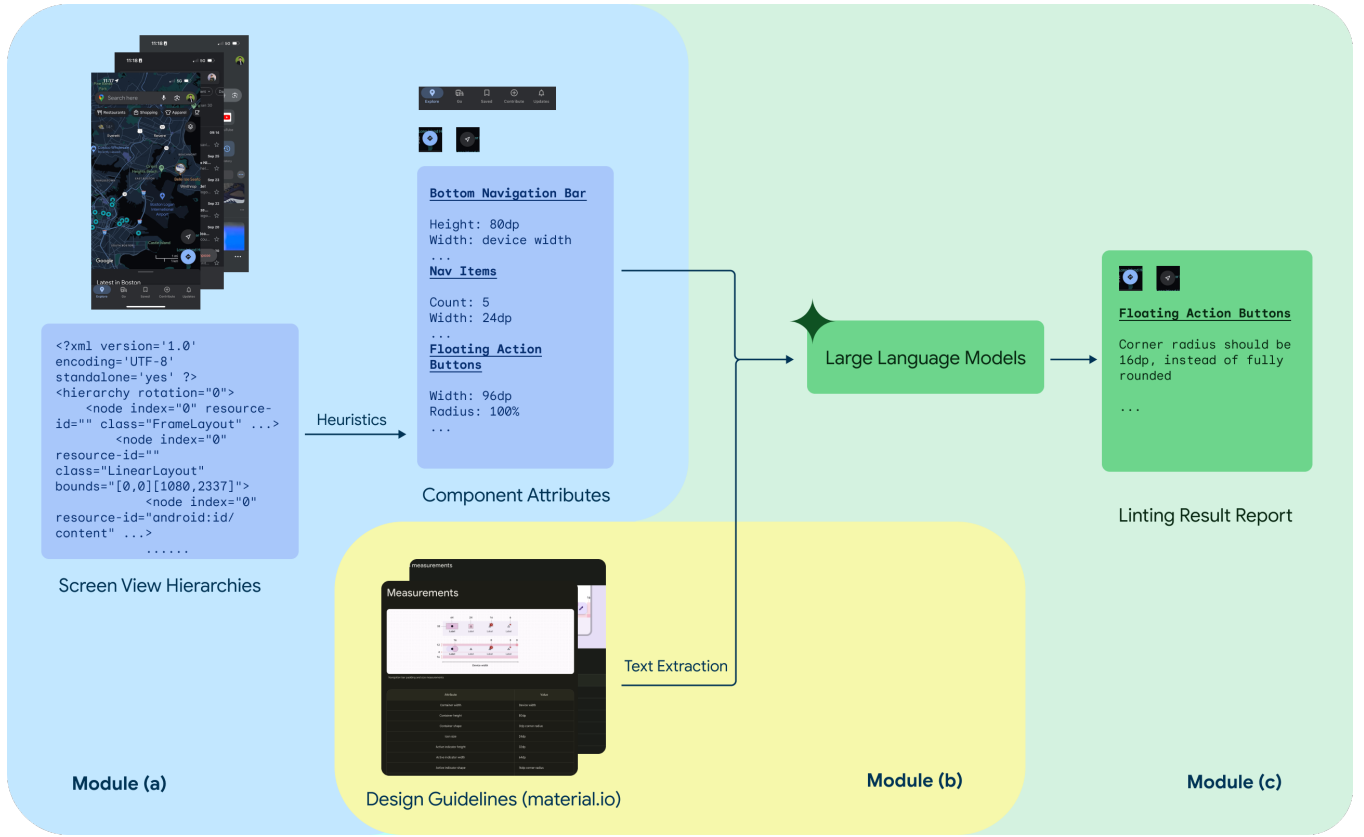


Figure 2: The design of our hybrid UI linter pipeline.

Module (c) utilizes the flexibility and emergent reasoning ability of LLMs. By comparing the element attributes and design guidelines side-by-side, LLMs can conduct design linting at a granular level (DN2) with high alignment with design systems (DN3). LLMs can also flexibly produce linting reports on the types of violations and improvement options. The specific format of UI linting outputs is a design task on its own and not the focus of our technical pipeline, so we intentionally leave it out for individual teams to decide.

In general, our hybrid technical pipeline addresses the needs of designers and supports granular scrutiny, flexibility, and scalability in UI linting. We believe that our pipeline can be a valuable tool for design teams to ensure the consistency and quality of their UI designs.

4 FUTURE CONSIDERATIONS

4.1 Unmet Needs in UI Linting with Current AI Models

As we discussed in section 2, modern-day AI models based on deep learning do not meet designers' specific needs in UI linting. While they might be good at versatile tasks like UI component detection [19] or UI-based question-answering [31], the uncertainty in reliably producing outputs that align with design systems constraint current models' adoption in real-world UI linting practices. Existing UI component detection models already support bounding

box prediction, but the results can contain inaccuracies [19, 23]. This makes them less useful in high-precision linting processes. Our hybrid solution design is a workaround given the constraints we have, but in the future, when high-quality UI linting datasets are available, it is possible to train specific models or finetune large pre-trained models for UI linting.

4.2 Current Limitations on Less Straightforward Design Guidelines

Our hybrid pipeline design is limited to the most straightforward design guidelines, such as component size and position, and color contrast. Violation detections that require more contextual information, such as wrong component selection for different information display purposes⁷, our designed solution might be constrained. Potential solutions lie in either a more flexible prompt design on top of our proposed approach, or a vision-based, deep learning approach built on high-quality UI linting datasets that contain relevant instances.

4.3 Beyond Static UI

In our proposed approach, we conducted UI linting on the product implementation results, i.e. the view hierarchies that are rendered

⁷For the case of buttons versus chips, see <https://m3.material.io/components/chips/guidelines>

using implementation code. However, view hierarchies only represent static screenshots and cannot encapsulate user interactions, UI transitions, and motion design. These are essential design aspects that constitute important aspects of user experiences in modern digital products. In particular, given the novelty of motion design, many design violations occur in motion design [41]. Such interactions and animation data are available in the RICO dataset yet still under-utilized [9]. In the future, with more advancement in AI for dynamic UI understanding beyond static images, enabling automatic linting of dynamic UI animations and interactions is highly beneficial for design practitioners.

4.4 AI Adoption for Specific Use Cases: AI is not always the answer

In general, our case study demonstrated a scenario where AI models alone cannot effectively fulfill target users' practical needs. We showcased how pre-selecting AI as the solution can be ineffective for problem solving. Through rapid prototyping of potential technical solutions and soliciting feedback from designers, we were able to pinpoint their critical unmet needs with current AI models (section 2.2). By combining deterministic, heuristics-based modules with LLMs, we were able to design a technical pipeline that meets designers' needs and utilizes the latest AI models' impressive abilities. We argue that human-centered design methodologies and a good understanding of AI's technical capabilities will play an important role in ongoing AI adoption. It is important to focus on the value AI can provide, instead of blindly throwing AI models at problems they might not be suitable for.

5 CONCLUSION

In this case study, we investigated the potential of AI integration in UI linting. Through rapid prototyping of potential technical solutions and collaborations with UX designers, we identify the limitations of current AI models in UI linting. We also pinpointed designers' core needs in practical linting practices. Based on these insights, we designed and presented a hybrid technical pipeline that combined the reliability of heuristics and the flexibility of LLMs. Our case study demonstrated why and how AI alone may not be enough for real-world problems, and how a deep understanding of AI capabilities and user-centered design approaches can support effective AI adoption.

ACKNOWLEDGMENTS

We want to thank Toby Li, Kunal Patel, Brandon Keely, Kenichi Yoneda, Gang Li, Yang Li, Forrest Huang, Chris Fay, and Ruofei Du for providing feedback and inspiration for our work. We also appreciate Po-Yuan Teng and Gilles Baechler for providing technical support in our exploration of potential technical solutions.

REFERENCES

- [1] [n.d.]. Layout – Material Design 3. <https://m3.material.io/foundations/layout/understanding-layout/spacing>. Accessed: 2023-9-27.
- [2] Gary Ang and Ee-Peng Lim. 2022. Learning Semantically Rich Network-based Multi-modal Mobile User Interface Embeddings. *ACM Transactions on Interactive Intelligent Systems* 12, 4 (Dec. 2022), 1–29.
- [3] Diego Martin Arroyo, Janis Postels, and Federico Tomba. 2021. Variational Transformer Networks for Layout Generation.
- [4] Emil Borjesson and Robert Feldt. 2012. Automated System Testing Using Visual GUI Testing Tools: A Comparative Study in Industry. In *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*. 350–359.
- [5] Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. (May 2020). arXiv:2005.14165 [cs.CL]
- [6] Jieshan Chen, Amanda Swearngin, Jason Wu, Titus Barik, Jeffrey Nichols, and Xiaoyi Zhang. 2022. Extracting Replayable Interactions from Videos of Mobile App Usage.
- [7] Jieshan Chen, Amanda Swearngin, Jason Wu, Titus Barik, Jeffrey Nichols, and Xiaoyi Zhang. 2022. Towards Complete Icon Labeling in Mobile Applications. In *CHI Conference on Human Factors in Computing Systems*. ACM, New Orleans LA USA, 1–14.
- [8] Elizabeth F Churchill. 2019. Scaling UX with design systems. *Interactions* 26, 5 (Aug. 2019), 22–23.
- [9] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschan, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*. ACM, Québec City QC Canada, 845–854.
- [10] K J Kevin Feng, Tony W Li, and Amy X Zhang. 2023. Understanding Collaborative Practices and Tools of Professional UX Practitioners in Software Organizations. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*. Association for Computing Machinery, New York, NY, USA, 1–20.
- [11] Weixi Feng, Wanrong Zhu, Tsu-Jui Fu, Varun Jampani, Arjun Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and William Yang Wang. 2023. LayoutGPT: Compositional Visual Planning and Generation with Large Language Models.
- [12] Hiroki Furuta, Ofir Nachum, Kuang-Huei Lee, Yutaka Matsuo, Shixiang Shane Gu, and Izzeddin Gur. 2023. Instruction-Finetuned Foundation Models for Multimodal Web Navigation. (2023).
- [13] Anisha Gunjal, Jihan Yin, and Erhan Bas. 2023. Detecting and Preventing Hallucinations in Large Vision Language Models. (Aug. 2023). arXiv:2308.06394 [cs.CV]
- [14] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems (CHI '99)*. Association for Computing Machinery, New York, NY, USA, 159–166.
- [15] Ellen Jiang, Kristen Olson, Edwin Toh, Alejandra Molina, Aaron Donsbach, Michael Terry, and Carrie J Cai. 2022. PromptMaker: Prompt-based Prototyping with Large Language Models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, 1–8.
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-Tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. (May 2020). arXiv:2005.11401 [cs.CL]
- [17] Toby Jia-Jun Li, Lindsay Popowski, Tom Mitchell, and Brad A Myers. 2021. Screen2Vec: Semantic Embedding of GUI Screens and GUI Components. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–15.
- [18] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. 2023. Evaluating Object Hallucination in Large Vision-Language Models. (May 2023). arXiv:2305.10355 [cs.CV]
- [19] Yang Li, Gang Li, Xin Zhou, Mostafa Dehghani, and Alexey Gritsenko. 2021. VUT: Versatile UI Transformer for Multi-Modal Multi-Task User Interface Modeling.
- [20] Tony Lindeberg. 2012. Scale invariant feature transform. 7, 5 (2012), 10491.
- [21] Yuwen Lu, Chengzhi Zhang, Iris Zhang, and Toby Jia-Jun Li. 2022. Bridging the Gap Between UX Practitioners' Work Practices and AI-Enabled Design Support Tools. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems (CHI EA '22)*. Association for Computing Machinery, New York, NY, USA, 1–7.
- [22] Chuan Luo, Wei Yang, Panling Huang, and Jun Zhou. 2019. Overview of Image Matching Based on ORB Algorithm. *J. Phys. Conf. Ser.* 1237, 3 (June 2019), 032020.
- [23] Dipu Manandhar, Hailin Jin, and John Collomosse. 2021. Magic Layouts: Structural Prior for Component Detection in User Interface Designs. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, Nashville, TN, USA, 15804–15813.
- [24] Lucas Memmert and Navid Tavanapour. 2023. Towards Human-AI-Collaboration in Brainstorming: Empirical Insights into the Perception of Working with a Generative AI. (2023).
- [25] Seokhyeon Park, Wonjae Kim, Young-Ho Kim, and Jinwook Seo. 2023. Computational Approaches for App-to-App Retrieval and Design Consistency Check. (Sept. 2023). arXiv:2309.10328 [cs.HC]
- [26] Emma J Rose, Craig M Macdonald, and Cynthia Putnam. 2022. Teaching Design Systems: Towards a flexible and scalable model for the UX classroom. In *Proceedings of the 40th ACM International Conference on Design of Communication*

- (Boston, MA, USA) (*SIGDOC '22*). Association for Computing Machinery, New York, NY, USA, 107–113.
- [27] Vinoth Pandian Sermuga Pandian, Sarah Suleri, Christian Beecks, and Matthias Jarke. 2021. MetaMorph: AI Assistance to Transform Lo-Fi Sketches to Higher Fidelities. In *Proceedings of the 32nd Australian Conference on Human-Computer Interaction (OzCHI '20)*. Association for Computing Machinery, New York, NY, USA, 403–412.
 - [28] Vinoth Pandian Sermuga Pandian, Sarah Suleri, and Matthias Jarke. 2021. SynZ: Enhanced Synthetic Dataset for Training UI Element Detectors. In *26th International Conference on Intelligent User Interfaces*. ACM, College Station TX USA, 67–69.
 - [29] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, stochastic model-based GUI testing of Android apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (Paderborn, Germany) (ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 245–256.
 - [30] Sarrah Vesselov and Taurie Davis. 2019. *Building Design Systems*. Apress.
 - [31] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling Conversational Interaction with Mobile UI using Large Language Models.
 - [32] Bryan Wang, Gang Li, Xin Zhou, Zhouong Chen, Tovi Grossman, and Yang Li. 2021. Screen2Words: Automatic Mobile UI Summarization with Multimodal Learning.
 - [33] Junyang Wang, Yiyang Zhou, Guohai Xu, Pengcheng Shi, Chenlin Zhao, Haiyang Xu, Qinghao Ye, Ming Yan, Ji Zhang, Jihua Zhu, Jitao Sang, and Haoyu Tang. 2023. Evaluation and Analysis of Hallucination in Large Vision-Language Models. (Aug. 2023). arXiv:2308.15126 [cs.LG]
 - [34] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2023. Empowering LLM to use Smartphone for Intelligent Task Automation.
 - [35] Thomas D White, Gordon Fraser, and Guy J Brown. 2019. Improving random GUI testing with image-based widget detection. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (Beijing, China) (ISSTA 2019)*. Association for Computing Machinery, New York, NY, USA, 307–317.
 - [36] Jason Wu, Amanda Swearngin, Xiaoyi Zhang, Jeffrey Nichols, and Jeffrey P Bigham. 2023. Screen Correspondence: Mapping Interchangeable Elements between UIs.
 - [37] Jason Wu, Siyan Wang, Siman Shen, Yi-Hao Peng, Jeffrey Nichols, and Jeffrey P Bigham. 2023. WebUI: A Dataset for Enhancing Visual UI Understanding with Web Semantics.
 - [38] Peng Xu, Wenqi Shao, Kaipeng Zhang, Peng Gao, Shuo Liu, Meng Lei, Fanqing Meng, Siyuan Huang, Yu Qiao, and Ping Luo. 2023. LVLm-eHub: A Comprehensive Evaluation Benchmark for Large Vision-Language Models. (June 2023). arXiv:2306.09265 [cs.CV]
 - [39] Bo Yang, Zhenchang Xing, Xin Xia, Chunyang Chen, Deheng Ye, and Shanping Li. 2021. Don't Do That! Hunting Down Visual Design Smells in Complex UIs Against Design Guidelines. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. 761–772.
 - [40] Xiaoyi Zhang, Lilian De Greef, Amanda Swearngin, Samuel White, Kyle Murray, Lisa Yu, Qi Shan, Jeffrey Nichols, Jason Wu, Chris Fleizach, Aaron Everitt, and Jeffrey P Bigham. 2021. Screen Recognition: Creating Accessibility Metadata for Mobile Applications from Pixels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, Yokohama Japan, 1–15.
 - [41] Dehai Zhao, Zhenchang Xing, Chunyang Chen, Xiwei Xu, Liming Zhu, Guoqiang Li, and Jinshui Wang. 2020. Seenomaly: vision-based linting of GUI animation effects against design-don't guidelines. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*. Association for Computing Machinery, New York, NY, USA, 1286–1297.